Database Business Documentation

MUSEUM MANAGEMENT SYSTEM

CONTENTS

1. BUSINESS DESCRIPTION

1.1 PROJECT OVERVIEW

The Museum Management System is a relational database designed to digitize and centralize the core operational data of a museum. It manages information about visitors, guides, exhibitions, museum items, and inventory– providing a unified data layer for administration and reporting.

1.2 PROJECT VISION

During the development of this database, I started from the business idea of digitizing information about registered visitors, their visits to exhibitions, as well as the museum's collection items and their storage locations. Based on this idea, I was able to identify the main entities at the conceptual modeling stage, as well as additional associative entities required to implement many-to-many relationships.

The attributes of most entities are straightforward and self-explanatory. However, it's important to say, that in the current implementation, the pricing model is simplified. There are only two ticket prices: 10 and 15 dollars, representing discounted (e.g., student) and standard (adult) tickets, while admission for children is assumed to be free. This information is implicitly reflected only in the Visits table through the total price attribute. In future development, it would be more appropriate to introduce a separate Ticket type entity to explicitly represent different categories of tickets, their prices, and conditions.

Also of particular note is the Visit entity, which contains a date and time attribute. The use of the Timestamp data type reflects the fact that the time of the visit is recorded automatically from the terminal.

In contrast, the Exhibition entity contains only date attributes (no time). This design decision is based on the assumption that the museum operates on the same schedule every day, so for planning purposes it is sufficient to store only the start and end dates. Since exhibitions are planned in advance, null values are not allowed for these attributes.

Regarding museum items, the current model includes only basic descriptive information. In the future, it is planned to extend the database by adding a separate entity for authors or creators of exhibits and their related details, which would improve data normalization and expand functionality.

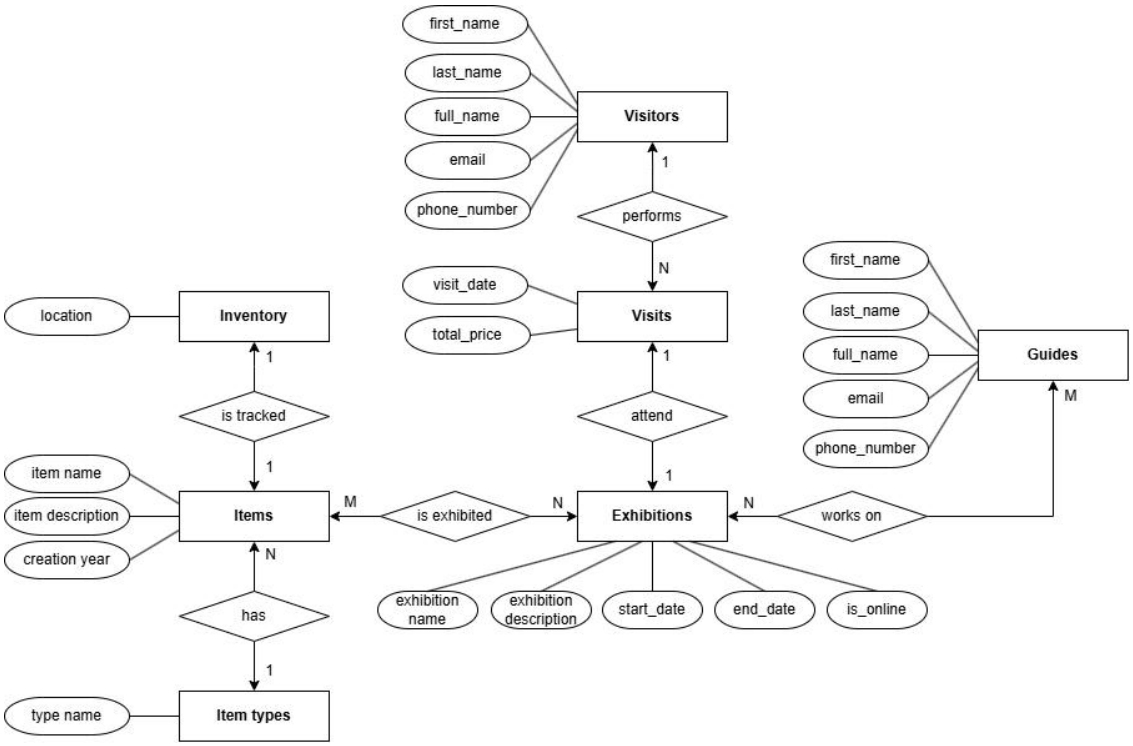
Museum staff in this database are represented as guides or specialists who can assist visitors during specific exhibitions or conduct full guided tours. As a potential extension, the system could be enhanced by introducing an entity for planned tours, allowing visitors to select guides and schedule personalized exhibitions.

2. MODEL DESCRIPTION

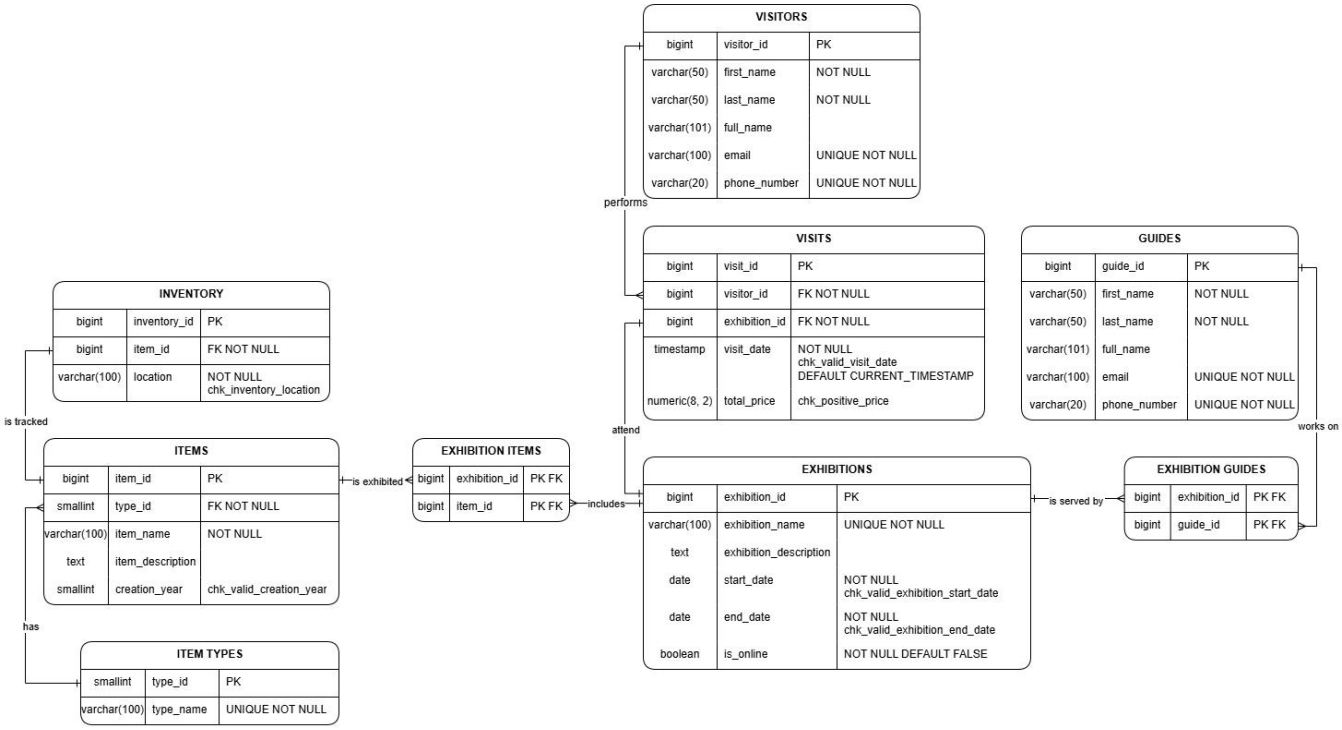
2.1 DEFINITIONS & ACRONYMS

Term / Acronym	Definition
PK	Primary Key- a column or set of columns that uniquely identifies each row in a table.
FK	Foreign Key- a column that references the PK of another table, enforcing referential integrity.
1NF / 2NF / 3NF	First, Second, and Third Normal Forms- levels of database normalization to eliminate redundancy.
Visitor	Any adult person who attends a museum exhibition, either registered in the system or not. Let's assume that admission for children is free and no registration is kept.
Guide	A museum employee that can be hired to conduct one or more exhibitions.
Exhibition	A curated collection of items displayed over a defined date range, either on-site or online.
Item	A museum artefact, artwork, document, or object belonging to the museum collection.
Inventory	A record of the physical location of a museum item within the building.
Visit	A transactional record of one visitor attending one exhibition on a specific date and time.
Is_online	Boolean flag indicating whether an exhibition is available for online/virtual attendance.

2.2 CONCEPTUAL SCHEME



2.3 LOGICAL SCHEME



3. OBJECTS

Each table in the Museum Management System represents a distinct real-world concept or event. The following sections describe each table, its purpose, normalization level, key constraints, and sample data.

3.1 VISITORS

The VISITORS table represents any individual who can attend museum exhibitions. It stores personal identification data (name, email, phone) and serves as the person entity in the system. A special GUEST record (guest@museum.com) acts as the anonymous identity for unregistered walk-in visitors, allowing visit tracking without requiring personal data collection.

Why must this information be stored separately?

Visitor data is stored separately from visits to avoid repetition of personal data across every visit record, to enforce uniqueness of contact details (email, phone), and to allow a visitor to be updated (e.g., change of email) in one place. Merging it into visits would violate 1NF and 2NF.

Normalization level:

3NF. The primary key is visitor_id (surrogate). first_name, last_name, email, and phone_number all depend solely on the key. The derived column full_name is a computed/generated column and introduces no transitive dependency.

Primary Key / Foreign Keys:

PK: visitor_id (BIGINT, auto-generated). No FK references in this table. visitor_id is referenced as FK in the visits table.

Table Structure:

Table Name	Field Name	Field Description	Data Type
visitors	visitor_id	Unique identifier for each visitor	BIGINT PK
	first_name	Visitor's first name	VARCHAR(50) NOT NULL
	last_name	Visitor's last name	VARCHAR(50) NOT NULL
	full_name	Computed full name (generated column)	VARCHAR(101)
	email	Visitor's email address	VARCHAR(100) UNIQUE NOT NULL

	phone_number	Visitor's phone number	VARCHAR(20) UNIQUE NOT NULL

Example Data:

visitor_id	first_name	last_name	email	phone_number
1	GUEST	GUEST	guest@museum.com	0000000000
2	Robert	Smith	r.smith@email.com	+1555010222
3	Emily	Davis	emily.d@email.com	+1555010333
4	Sophia	Brown	sophia.b@email.com	+1555010555

3.2 GUIDES

The GUIDES table stores information about museum employees who lead and manage exhibitions. Each guide has a unique identity (name, email, phone) and can be assigned to one or more exhibitions.

Why must this information be stored separately?

Guides are stored separately from exhibitions to avoid duplicating guide personal data for every exhibition they manage. Separation follows the same normalization logic as VISITORS and enables guides to be updated or retired independently of the exhibitions they have worked on.

Normalization level:

3NF. The primary key is guide_id. All non-key attributes (first_name, last_name, email, phone_number, full_name) depend only on guide_id with no transitive dependencies.

Primary Key / Foreign Keys:

PK: guide_id (BIGINT, auto-generated). guide_id is referenced as FK in the exhibition_guides junction table.

Table Structure:

Table Name	Field Name	Field Description	Data Type
guides	guide_id	Unique identifier for each guide	BIGINT PK
	first_name	Guide's first name	VARCHAR(50) NOT NULL
	last_name	Guide's last name	VARCHAR(50) NOT NULL

	full_name	Computed full name (generated column)	VARCHAR(101)
	email	Guide's work email address	VARCHAR(100) UNIQUE NOT NULL
	phone_number	Guide's work phone number	VARCHAR(20) UNIQUE NOT NULL

Example Data:

guide_id	first_name	last_name	email	phone_number
1	James	Anderson	j.anderson@museum.org	+1555090111
2	Linda	Taylor	l.taylor@museum.org	+1555090222
3	Barbara	Moore	b.moore@museum.org	+1555090444

3.3 EXHIBITIONS

The EXHIBITIONS table represents curated displays of museum items offered to the public over a defined time period. Each exhibition has a unique name, descriptive text, date range, and a flag indicating whether it is available online. Exhibitions are the central organizational unit linking visitors (via visits), items (via exhibition_items), and guides (via exhibition_guides).

Why must this information be stored separately?

Exhibition data is stored separately because it is a distinct entity with its own lifecycle (start and end dates), its own staff, and its own item collection. Embedding exhibition data in visits or items would cause massive duplication and make date-range queries and scheduling impossible.

Normalization level:

3NF. The primary key is exhibition_id. All attributes (exhibition_name, description, start_date, end_date, is_online) depend fully and only on exhibition_id.

Primary Key / Foreign Keys:

PK: exhibition_id (BIGINT, auto-generated). exhibition_id is referenced as FK in visits, exhibition_items, and exhibition_guides.

Table Structure:

Table Name	Field Name	Field Description	Data Type

exhibitions	exhibition_id	Unique identifier for each exhibition	BIGINT PK
	exhibition_name	Name of the exhibition	VARCHAR(100) UNIQUE NOT NULL
	exhibition_description	Detailed description of the exhibition	TEXT
	start_date	Start date; must be >= 2026-01-01	DATE NOT NULL
	end_date	End date; must be > start_date	DATE NOT NULL
	is_online	Whether the exhibition is available online	BOOLEAN NOT NULL DEFAULT FALSE

Example Data:

exhibition_id	exhibition_name	start_date	end_date	is_online
1	Masterpieces of World Art	2026-02-15	2026-04-20	false
2	History and Science Through Time	2026-02-01	2026-12-31	true
3	Ancient Civilizations	2026-03-10	2026-06-15	false

3.4 VISITS

The VISITS table is a transactional entity recording each instance of a visitor attending an exhibition. It captures the precise timestamp of the visit and the price paid. It acts as a fact table, connecting the visitor and exhibition dimensions and supporting revenue and attendance analytics.

Why must this information be stored separately?

Visits must be stored separately because they are events– distinct occurrences that happen at a point in time. Embedding visit data inside the visitor or exhibition tables would create repeating groups, violate 1NF, and make it impossible to track multiple visits by the same person to different exhibitions.

Normalization level:

3NF. The composite relationship (visitor_id, exhibition_id, visit_date) naturally identifies each row. total_price depends on the visit event itself and not on either FK independently.

Primary Key / Foreign Keys:

PK: visit_id (BIGINT, auto-generated). FK: visitor_id references visitors(visitor_id); FK: exhibition_id references exhibitions(exhibition_id).

Table Structure:

Table Name	Field Name	Field Description	Data Type
visits	visit_id	Unique identifier for each visit	BIGINT PK
	visitor_id	Reference to the visiting person	BIGINT FK NOT NULL
	exhibition_id	Reference to the attended exhibition	BIGINT FK NOT NULL
	visit_date	Timestamp of the visit; must be >= 2026-01-01; defaults to now	TIMESTAMP NOT NULL
	total_price	Price paid for the visit; must be >= 0	NUMERIC(8,2)

Example Data:

visit_id	visitor_id	exhibition_id	visit_date	total_price
1	1 (GUEST)	1 (Masterpieces of World Art)	2026-02-07 12:38:59	10.00
2	3 (Emily Davis)	1 (Masterpieces of World Art)	2026-03-12 14:21:16	15.00
3	4 (Sophia Brown)	2 (History and Science)	2026-04-23 18:09:41	15.00

3.5 ITEM_TYPES

The ITEM_TYPES table is a reference (lookup) table that classifies museum items into categories such as Artwork, Sculpture, Artifact, Manuscript, Scientific Instrument, and so on. It standardizes item classification and prevents free-text inconsistencies.

Why must this information be stored separately?

Storing types in a dedicated table rather than as a plain text column in items allows: consistent spelling and validation via FK, easy extension with new types, and efficient filtering/grouping of items by type without string comparison.

Normalization level:

1NF / 2NF / 3NF. The table has a simple PK (type_id) and a single non-key attribute (type_name), which depends entirely on type_id. There are no partial or transitive dependencies.

Primary Key / Foreign Keys:

PK: type_id (SMALLINT, manually assigned). type_id is referenced as FK in the items table.

Table Structure:

Table Name	Field Name	Field Description	Data Type
item_types	type_id	Unique identifier for the item type	SMALLINT PK
	type_name	Descriptive name of the item category	VARCHAR(100) UNIQUE NOT NULL

Example Data:

type_id	type_name
1	Artwork
2	Sculpture
4	Artifact
15	Scientific Instrument

3.6 ITEMS

The ITEMS table stores the museum's permanent collection. Each item is a unique physical or digital object (artwork, sculpture, manuscript, fossil, instrument, etc.) that can be displayed in exhibitions. Attributes include the item's name, description, type classification, and year of creation.

Why must this information be stored separately?

Items must be stored separately from exhibitions because an item has an existence independent of any particular exhibition– it was created in a specific year, belongs to a type, and can be featured in multiple exhibitions across time. Embedding items in exhibitions would make it impossible to track an item's full exhibition history.

Normalization level:

3NF. The primary key is item_id. item_name, item_description, creation_year, and type_id all depend only on item_id. type_id is a FK to item_types– not a transitive dependency, since it classifies the item without creating a chain of non-key dependencies.

Primary Key / Foreign Keys:

PK: item_id (BIGINT, auto-generated). FK: type_id references item_types(type_id). item_id is referenced as FK in inventory and exhibition_items.

Table Structure:

Table Name	Field Name	Field Description	Data Type
items	item_id	Unique identifier for each museum item	BIGINT PK
	type_id	Classification reference to item_types	SMALLINT FK NOT NULL
	item_name	Name of the museum item	VARCHAR(100) NOT NULL
	item_description	Descriptive text about the item	TEXT
	creation_year	Year the item was created (negative = BCE)	SMALLINT

Example Data:

item_id	type_id	item_name	creation_year
1	1 (Artwork)	Mona Lisa	1503
2	2 (Sculpture)	David	1504
4	4 (Artifact)	Rosetta Stone	-196
6	15 (Scientific Instrument)	Antikythera Mechanism	-100

3.7 INVENTORY

The INVENTORY table tracks the current physical location of each museum item within the building. Locations are constrained to known halls and rooms: Hall A, Hall B, Hall C, Storage, and Restoration Room. This enables staff to quickly locate any item in the collection.

Why must this information be stored separately?

Inventory is stored separately from items because location is an operational attribute that changes independently of the item's identity or type. An item may move between halls or enter storage without any change to its descriptive data. The separation enables accurate location tracking at any point in time.

Normalization level:

3NF. PK is inventory_id. item_id (FK) and location depend only on inventory_id. There are no transitive dependencies.

Primary Key / Foreign Keys:

PK: inventory_id (BIGINT, auto-generated). FK: item_id references items(item_id). Location is constrained by a CHECK to valid values: Hall A, Hall B, Hall C, Storage, Restoration Room.

Table Structure:

Table Name	Field Name	Field Description	Data Type
inventory	inventory_id	Unique identifier for each inventory record	BIGINT PK
	item_id	Reference to the museum item	BIGINT FK NOT NULL
	location	Physical location of the item in the museum	VARCHAR(100) NOT NULL CHECK(location IN ('Hall A','Hall B','Hall C','Storage','Restoration Room'))

Example Data:

inventory_id	item_id	item_name (ref)	location
1	1	Mona Lisa	Hall A
2	2	David	Hall A
3	3	The Last Supper Sketches	Hall B
4	4	Rosetta Stone	Hall C

3.8 EXHIBITION_ITEMS

The EXHIBITION_ITEMS junction table resolves the many-to-many relationship between exhibitions and items. Each row records that a specific item is (or was) displayed in a specific exhibition.

Why must this information be stored separately?

This junction table is necessary because a single exhibition can feature many items, and a single item can be featured in multiple exhibitions over time. Without this table, either side would require multi-valued attributes, violating 1NF.

Normalization level:

Composite PK (exhibition_id, item_id)– fully in 3NF with no non-key attributes at all.

Primary Key / Foreign Keys:

PK: composite (exhibition_id, item_id). FK: exhibition_id references exhibitions(exhibition_id); FK: item_id references items(item_id).

Table Structure:

Table Name	Field Name	Field Description	Data Type
exhibition_items	exhibition_id	Reference to the exhibition	BIGINT PK FK
	item_id	Reference to the displayed item	BIGINT PK FK

Example Data:

exhibition_id	exhibition_name (ref)	item_id	item_name (ref)
1	Masterpieces of World Art	1	Mona Lisa
1	Masterpieces of World Art	2	David
2	History and Science Through Time	4	Rosetta Stone
2	History and Science Through Time	6	Antikythera Mechanism

3.9 EXHIBITION_GUIDES

The EXHIBITION_GUIDES junction table records the assignment of guides to exhibitions. Each row establishes that a specific guide is responsible for servicing a specific exhibition.

Why must this information be stored separately?

A guide can work on multiple exhibitions, and an exhibition can have multiple guides– a classic many-to-many relationship. The junction table is the normalized solution, avoiding duplication of guide details in the exhibitions table or repetition of exhibition data in the guides table.

Normalization level:

Composite PK (exhibition_id, guide_id)– fully in 3NF with no non-key attributes.

Primary Key / Foreign Keys:

PK: composite (exhibition_id, guide_id). FK: exhibition_id references exhibitions(exhibition_id); FK: guide_id references guides(guide_id).

Table Structure:

Table Name	Field Name	Field Description	Data Type
exhibition_guides	exhibition_id	Reference to the exhibition	BIGINT PK FK
	guide_id	Reference to the assigned guide	BIGINT PK FK

Example Data:

exhibition_id	exhibition_name (ref)	guide_id	guide name (ref)
1	Masterpieces of World Art	1	James Anderson
1	Masterpieces of World Art	2	Linda Taylor
2	History and Science Through Time	4	Barbara Moore
2	History and Science Through Time	5	Richard Jackson

3.10 UPDATE_EXHIBITION_DATA (FUNCTION)

The update_exhibition_data function provides a controlled interface for modifying a single column in the exhibitions table. It accepts a surrogate key, a validated column name, and a new value as text, then applies the update at runtime using a dynamically constructed statement.

Purpose:

Administrative staff can correct or update exhibition metadata (name, description, dates, online flag) through a single reusable function rather than issuing direct UPDATE statements. All allowed columns are explicitly whitelisted inside the function body, so callers cannot target computed columns, surrogate keys, or columns that do not exist.

SQL-injection mitigation:

The function uses EXECUTE format(... %I ...) to safely quote the column identifier, combined with a parameterized USING clause for the value and the primary key. Because the value is passed as a bind parameter rather than interpolated with %L or %s, the database driver handles escaping entirely– no user-supplied string is ever concatenated directly into the SQL text.

Input arguments:

Parameter	Type	Description
p_exhibition_id	BIGINT	Surrogate key of the row to update
p_column_name	TEXT	Name of the column to update (whitelisted)

p_new_value	TEXT	New value to set, cast automatically to the column's data type

Allowed columns: exhibition_name, exhibition_description, start_date, end_date, is_online.

Error handling:

Raises EXCEPTION if the column name is not in the whitelist, if the value cannot be cast to the target data type, or if any other database error occurs.

A NOTICE is raised on success.

Example calls:

SELECT museum.update_exhibition_data(1, 'exhibition_description', 'Updated description for world masterpieces.');

SELECT museum.update_exhibition_data(2, 'end_date', '2027-01-01');

SELECT museum.update_exhibition_data(6, 'is_online', 'true');

3.11 NEW_VISIT (FUNCTION)

The new_visit function inserts a new record into the visits table. It accepts all transaction attributes via natural keys (email address and exhibition name) instead of surrogate IDs, resolving the corresponding IDs internally.

Purpose:

Encapsulating visit insertion in a function prevents direct manipulation of surrogate keys by calling code, enforces duplicate detection before the insert, and provides human-readable confirmation or error messages for front-end or scripting consumers.

Input arguments:

Parameter	Type	Description
p_email	VARCHAR(100)	Email address of the visiting person (natural key for visitors)
p_exhibition_name	VARCHAR(100)	Name of the exhibition (natural key for exhibitions)
p_visit_date	TIMESTAMP	Date and time of the visit
p_total_price	NUMERIC(8, 2)	Ticket price paid

Error handling: Raises EXCEPTION if the visitor email is not found, if the exhibition name is not found, or if an identical visit (same visitor, exhibition, and timestamp) already exists. Raises NOTICE on successful insertion.

Example call:

```
SELECT museum.new_visit(
    'emily.d@email.com',
    'History and Science Through Time',
    '2026-05-15 14:30:00'::TIMESTAMP,
```

```
    15.50
);
```

3.12 ANALYTICS_LAST_QTR (VIEW)

The analytics_last_qtr view computes a summary of museum activity for the most recently recorded quarter in the visits table. It is designed for management reporting and contains no surrogate keys or duplicate rows.

Purpose:

Rather than requiring managers to write aggregation queries, this view provides a ready-to-query snapshot of revenue, attendance, and exhibition metrics for the current period of interest. The target quarter is derived dynamically from the latest visit_date in the table, so the view remains accurate as new data is inserted without requiring manual date changes.

How the quarter is determined:

A CTE (last_quarter_info) truncates the maximum visit_date to the start of its calendar quarter using DATE_TRUNC('quarter', ...) and adds three months to compute the exclusive upper bound. All visits within that range are then aggregated.

Output columns:

Column	Description
target_year	Calendar year of the reported quarter
target_qtr	Quarter number (1–4)
total_revenue	Sum of all ticket prices paid in the quarter
active_exhibitions_count	Number of distinct exhibitions visited in the quarter
total_visits	Total number of visit records in the quarter
total_unique_visitors	Number of distinct visitors in the quarter
avg_visits_per_exhibition	Average number of visits per active exhibition (rounded to 2 d.p.)
avg_revenue_per_exhibition	Average revenue per active exhibition (rounded to 2 d.p.)

Example query:

```
SELECT * FROM museum.analytics_last_qtr;
```

3.13 MANAGER (READ-ONLY ROLE)

The manager role is a database-level login role granted read-only access to all tables in the museum schema. It follows the principle of least privilege: the role can connect to the database, traverse the schema, and run SELECT queries, but cannot modify any data or schema objects.

Permissions granted:

Privilege	Scope
CONNECT	Database museum
USAGE	Schema museum
SELECT	All tables in schema museum

Security notes:

The role is created conditionally (IF NOT EXISTS) to make the script safely re-runnable. The password is set at creation time and should be rotated in production using ALTER ROLE manager PASSWORD '...'. No INSERT, UPDATE, DELETE, TRUNCATE, or DDL privileges are granted, so a manager session cannot alter or destroy data even if credentials are compromised.

Example session:

```
SET ROLE manager;
SELECT * FROM museum.visits;
RESET ROLE;
```

4. HOW TO RUN

Prerequisites: PostgreSQL 14 or later; psql available on the command line; a superuser (or role with CREATEDB) for the initial database creation step.

Step 1– Create the database (admin connection required)

The CREATE DATABASE statement must be executed in a separate connection before running the main script, because psql cannot switch databases mid-file. Run this once in an admin session (e.g., connected to the default 'postgres' database):

CREATE DATABASE museum;

Step 2– Run the full script

psql -U postgres -d museum -f script.sql

Replace postgres with your superuser name if different. The -d museum flag connects to the database created in Step 1.

The script will:

- create the museum schema and all tables in correct DDL order (parent tables before child tables),
- add all foreign key relationships and CHECK constraints,
- insert sample data using idempotent WHERE NOT EXISTS guards,
- create the update_exhibition_data and new_visit functions,
- create the analytics_last_qtr view,
- create and configure the manager read-only role.

All DDL statements use IF NOT EXISTS and CREATE OR REPLACE where applicable, and all DML inserts are guarded with WHERE NOT EXISTS. The script is safe to run multiple times on the same database without producing errors or duplicate rows.